

Borrowing Polonius

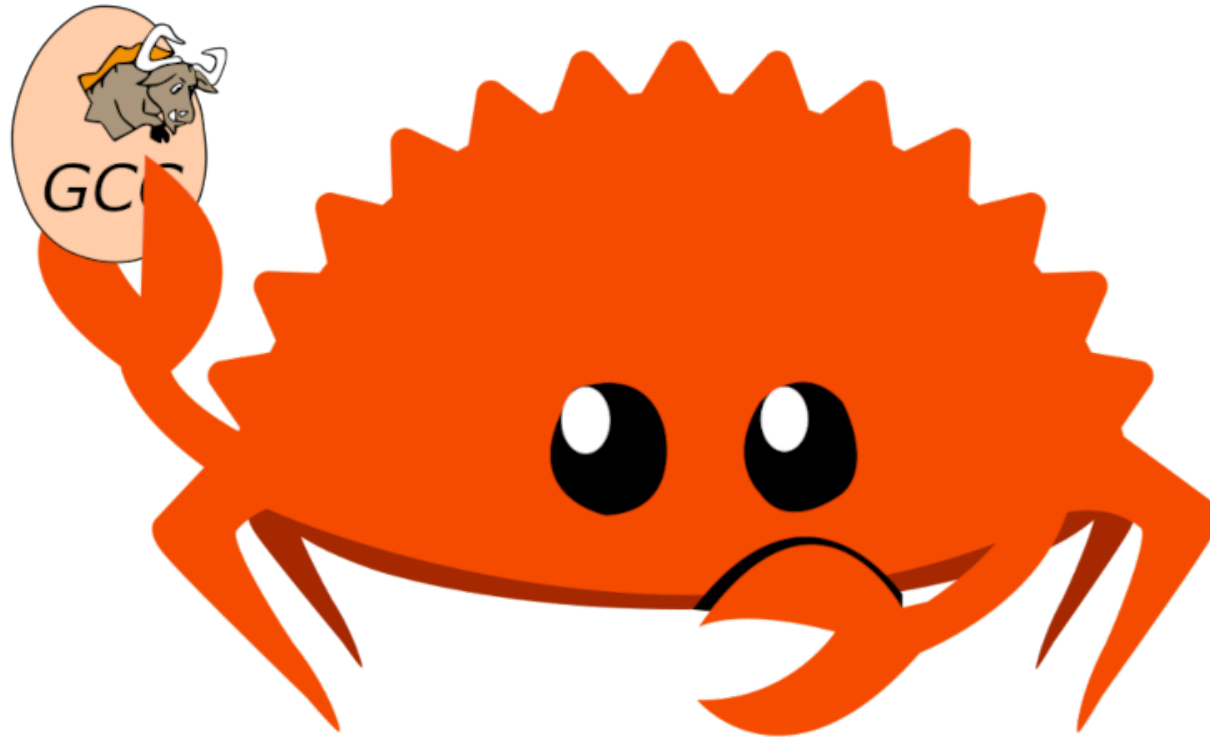
Jakub Dupák

GNU Tools Cauldron 2024

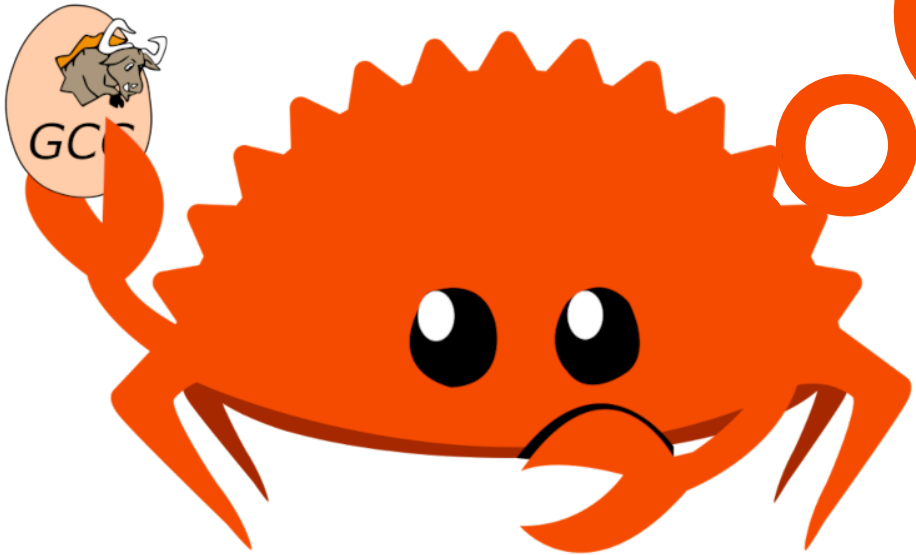
16. 9. 2024

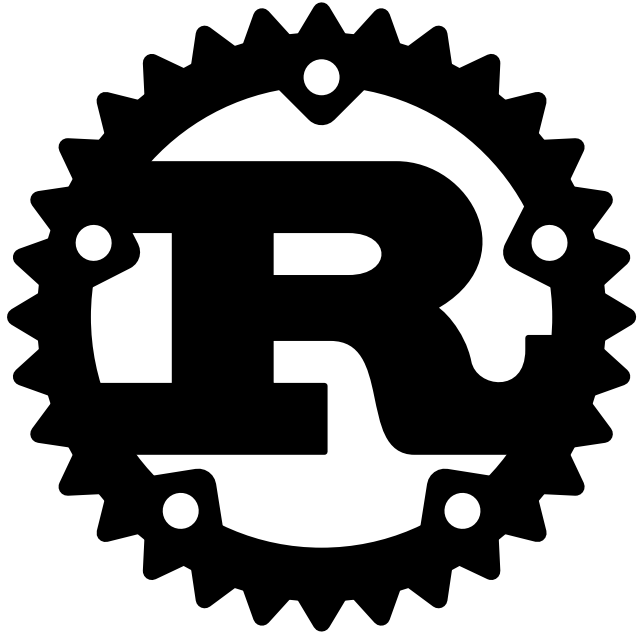
Rust?!!





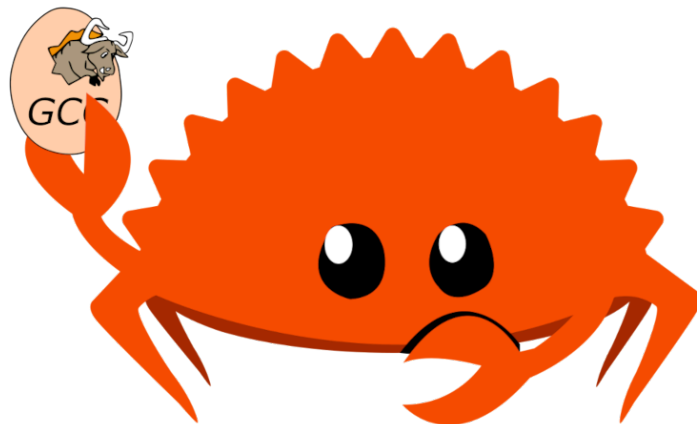
**Memory
safety???**





Polonius

&mut Polonius



Outline

- **About me**
- **Terminology**
- **Borrow checker rules**
- **History of borrow checking**
 - **Lexical**
 - **NLL**
 - **Polonius**
- **Rust GCC**
 - **BIR**
 - **Polonius engine**

About Me



Jakub Dupák

dev@jakubdupak.com

<https://jakubdupak.com>

About Me



Jakub Dupák

dev@jakubdupak.com

<https://jakubdupak.com>



Memory safety analysis in Rust GCC

FACULTY OF ELECTRICAL ENGINEERING

<https://dspace.cvut.cz/handle/10467/113390>

About Me



Jakub Dupák

dev@jakubdupak.com

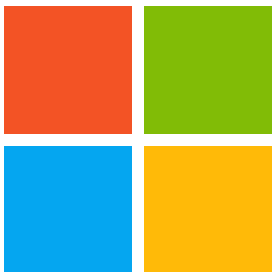
<https://jakubdupak.com>



Memory safety analysis in Rust GCC

FACULTY OF ELECTRICAL ENGINEERING

<https://dspace.cvut.cz/handle/10467/113390>



Software Engineer

RUST TOOLING GROUP

Microsoft Development Center Prague

Terminology

Reference | Borrow

checked pointer

let reference: &i32;

Borrowing

taking a reference

&value

Loan

the result of borrowing

&value

Lifetime | Origin | Region

***abstract notion of a part of
program***

range of lines, set of CFG nodes

Borrow Checker Rules

High-level Rules

§ 1

No invalid memory access

§ 2

No mutable reference aliasing

§ 1 **No invalid memory access**

- **Move**
- **Borrow must not outlive borrowee**
- **Lifetime subset relation**

§ 1 No invalid memory access

- Move

```
let mut v1 = Vec::new();  
v1.push(42)  
let mut v2 = v1; // <- Move  
println!(v1[0]); // <- Error
```

- Borrow must not outlive borrowee
- Lifetime subset relation

§ 1 No invalid memory access

- Move
- Borrow must not outlive borrowee
- Lifetime subset relation

```
fn f() -> &i32 {  
    &(1+1)  
} // <- Error
```

§ 2 **No mutable reference aliasing**

- **Either one mutable *live* reference or multiple (shared) immutable reference**
- **No modification of borrowed data**

§ 2 No mutable reference aliasing

- Either one mutable *live* reference or multiple (shared) immutable reference

```
let mut counter = 0;  
let ref1 = &mut counter;  
// ...  
let ref2 = &mut counter; // <- Error  
use(ref1)
```

- No modification of borrowed data

§ 2 No mutable reference aliasing

- Either one mutable *live* reference or multiple (shared) immutable reference

```
let mut counter = 0;  
let ref1 = &mut counter;  
// ...  
let ref2 = &mut counter; // <- Error  
use(ref1)
```

- No modification of borrowed data

Checking Functions

```
struct Vec<'a i32> { ... }
```

```
impl<'a> Vec<'a i32> {  
    fn push<'b>  
        where 'b: 'a (&mut self, x: &'b i32) {  
        // ...  
    }  
}
```

Checking Functions

```
struct Vec<'a i32> { ... }

impl<'a> Vec<'a i32> {
    fn push<'b>
        where 'b: 'a (&mut self, x: &'b i32) {
        // ...
    }
}
```

Checking Functions

```
struct Vec<'a i32> { ... }

impl<'a> Vec<'a i32> {
    fn push<'b>
        where 'b: 'a (&mut self, x: &'b i32) {
        // ...
    }
}
```

Propagating lifetimes

```
fn max_ref(a: &i32, b: &i32) -> &i32 {  
    let mut max = a;  
    if (*max < *b) {  
        max = b;  
    }  
    max  
}
```

Propagating lifetimes

```
fn max_ref(a: &'a i32, b: &'a i32)
  -> &'a i32 {
  let mut max = a;
  if (*max < *b) {
    max = b;
  }
  max
}
```

Propagating lifetimes

```
fn max_ref(a: &'a i32, b: &'b i32)
  -> &'c i32 {
  let mut max = a;
  if (*max < *b) {
    max = b;
  }
  max
}
```

Propagating lifetimes

```
fn max_ref(a: &'a i32, b: &'b i32)
  -> &'c i32 {
  let mut max: &i32 = a;
  if (*max < *b) {
    max = b;
  }
  max
}
```

Propagating lifetimes

```
fn max_ref(a: &'a i32, b: &'b i32)
  -> &'c i32 {
  let mut max: &'?1 i32 = a;
  if (*max < *b) {
    max = b;
  }
  max
}
```

max = a	'a: ' ?1
max = b	'b: ' ?1
return max	' ?1: ' c

Borrow checker evolution

Lexical, NLL, Polonius

Lexical borrow checker

lifetime = lexical scope*

Lexical borrow checker

```
fn foo() {  
    let mut data = vec!['a', 'b', 'c'];  
    capitalize(&mut data[..]);  
    data.push('d');  
    data.push('e');  
    data.push('f');  
}  
  
fn capitalize(data: &mut [char]) {  
    // do something  
}
```

Lexical borrow checker

```
fn foo() {  
    let mut data = vec!['a', 'b', 'c'];  
    capitalize(&mut data[..]);  
    data.push('d');  
    data.push('e');  
    data.push('f');  
}  
  
fn capitalize(data: &mut [char]) {  
    // do something  
}
```

Lexical borrow checker

```
fn foo() {  
    let mut data = vec!['a', 'b', 'c'];  
    capitalize(&mut data[..]);  
    data.push('d');  
    data.push('e');  
    data.push('f');  
}  
  
fn capitalize(data: &mut [char]) {  
    // do something  
}
```

Lexical borrow checker

```
fn foo() {  
    let mut data = vec!['a', 'b', 'c'];  
    capitalize(&mut data[..]);  
    data.push('d');  
    data.push('e');  
    data.push('f');  
}
```

```
fn capitalize(data: &mut [char]) {  
    // do something  
}
```

Lexical borrow checker

```
fn foo() {  
    let mut data = vec!['a', 'b', 'c']; // --+  
    capitalize(&mut data[..]);          // |  
    // ^~~~~~ 'lifetime // |  
    data.push('d');                      // |  
    data.push('e');                      // |  
    data.push('f');                      // |  
} // <-----+  
  
fn capitalize(data: &mut [char]) {  
    // do something  
}
```

Lexical borrow checker

```
fn bar() {  
    let mut data = vec!['a', 'b', 'c'];  
    let slice = &mut data[..]; // <-+ 'lifetime  
    capitalize(slice);          //      |  
    data.push('d'); // ERROR!    //      |  
    data.push('e'); // ERROR!    //      |  
    data.push('f'); // ERROR!    //      |  
} // <-----+
```


Lexical borrow checker

```
fn bar() {  
    let mut data = vec!['a', 'b', 'c'];  
    {  
        let slice = &mut data[..]; // <-+  
        capitalize(slice);          //   |  
    } // <-----+  
    data.push('d'); // OK  
    data.push('e'); // OK  
    data.push('f'); // OK  
}
```

Lexical borrow checker

```
fn process_or_default<K,V:Default>(
  map: &mut HashMap<K,V>,
  key: K
) {
  match map.get_mut(&key) { // -----+
    Some(value) => process(value), // |
    None => { // |
      map.insert(key, V::default()); // |
      // ^~~~~~ ERROR. // |
    } // |
  } // |
} // <-----+
}
```

Lexical borrow checker

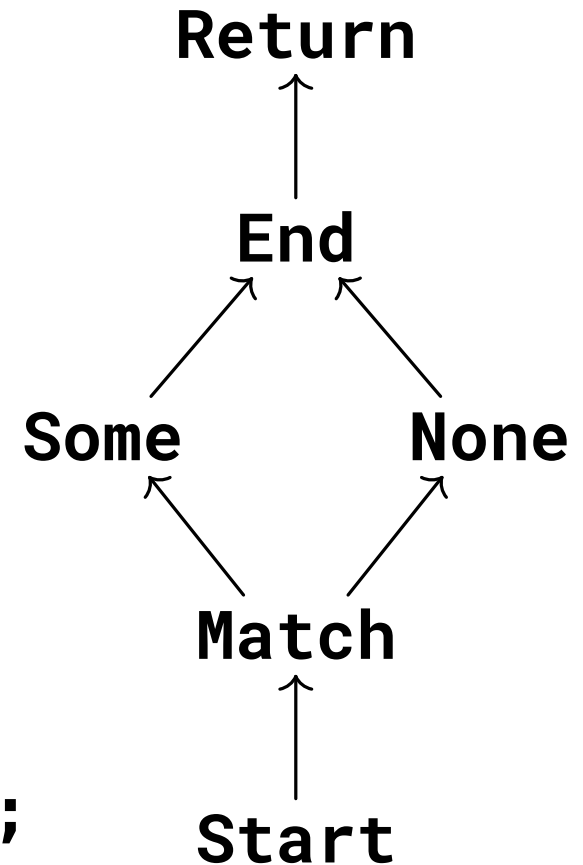
```
fn process_or_default<K,V:Default>(
  map: &mut HashMap<K,V>,
  key: K
) {
  match map.get_mut(&key) { // -----+
    Some(value) => process(value), // |
    None => { // |
      map.insert(key, V::default()); // |
      // ^~~~~~ ERROR. // |
    } // |
  } // |
} // <-----+
}
```

Non-lexical lifetimes (NLL)

lifetime = set of CFG nodes

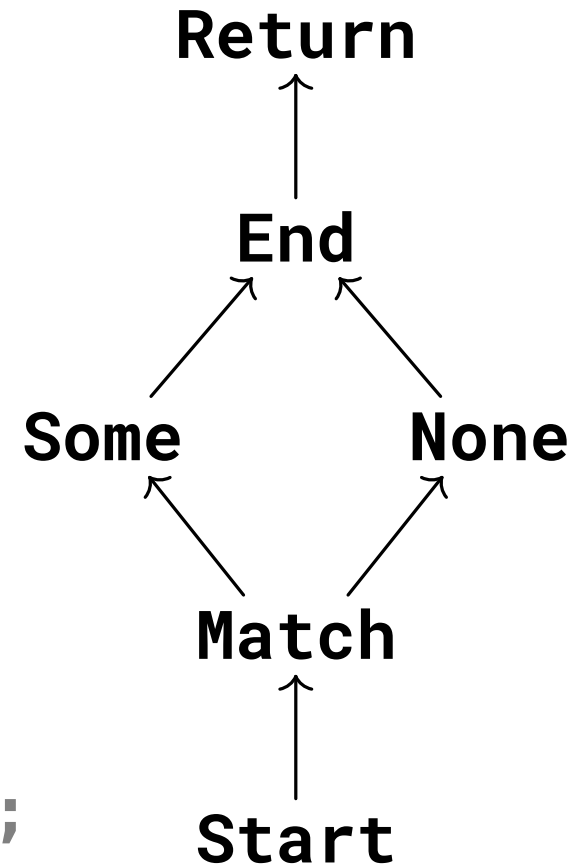
Non-lexical lifetimes (NLL)

```
fn process_or_default<K, V>(
    map: &mut HashMap<K, V>,
    key: K
) {
    match map.get_mut(&key) {
        Some(value) => {
            process(value);
        }
        None =>
    {
        map.insert(key, ...);
    }
}
}
```



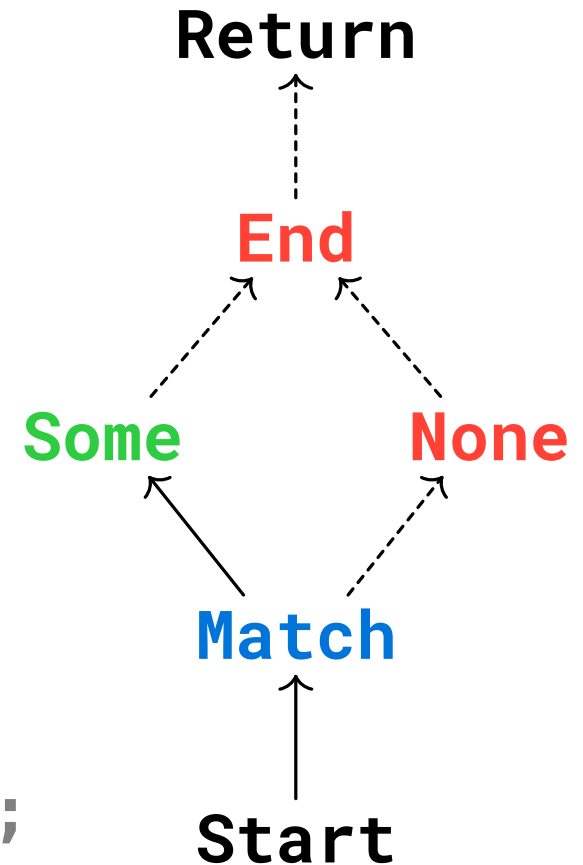
Non-lexical lifetimes (NLL)

```
fn process_or_default<K, V>(
    map: &mut HashMap<K, V>,
    key: K
) {
    match map.get_mut(&key) {
        Some(value) => {
            process(value);
        }
        None =>
    {
        map.insert(key, ...);
    }
}
}
```



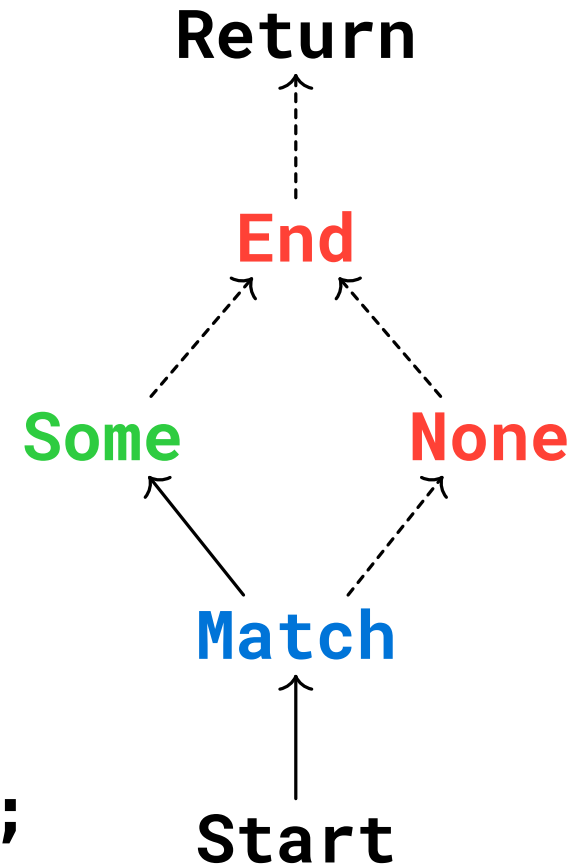
Non-lexical lifetimes (NLL)

```
fn process_or_default<K, V>(
    map: &mut HashMap<K, V>,
    key: K
) {
    match map.get_mut(&key) {
        Some(value) => {
            process(value);
        }
        None =>
    {
        map.insert(key, ...);
    }
}
}
```



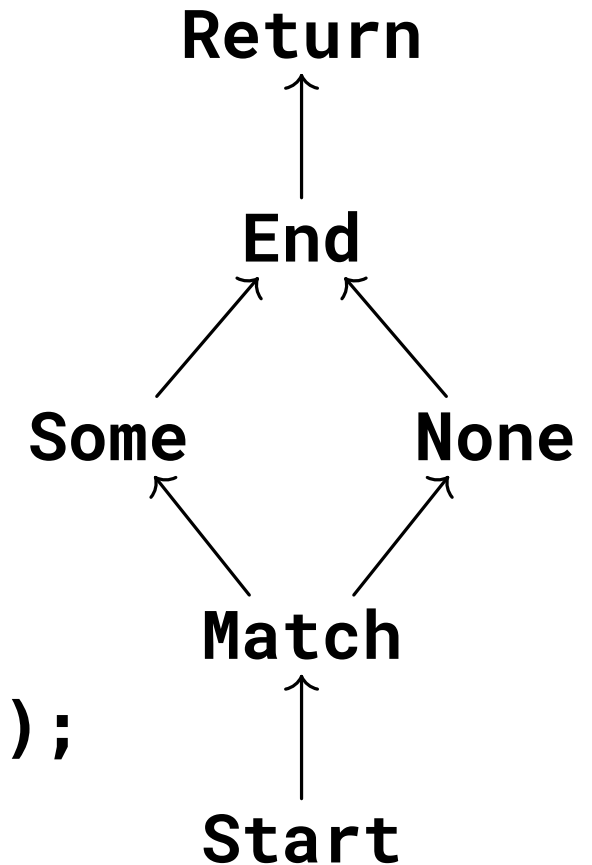
Non-lexical lifetimes (NLL)

```
fn process_or_default<K, V>(
    map: &mut HashMap<K, V>,
    key: K
) {
    match map.get_mut(&key) {
        Some(value) => {
            process(value);
        }
        None =>
    {
        map.insert(key, ...);
    }
}
}
```



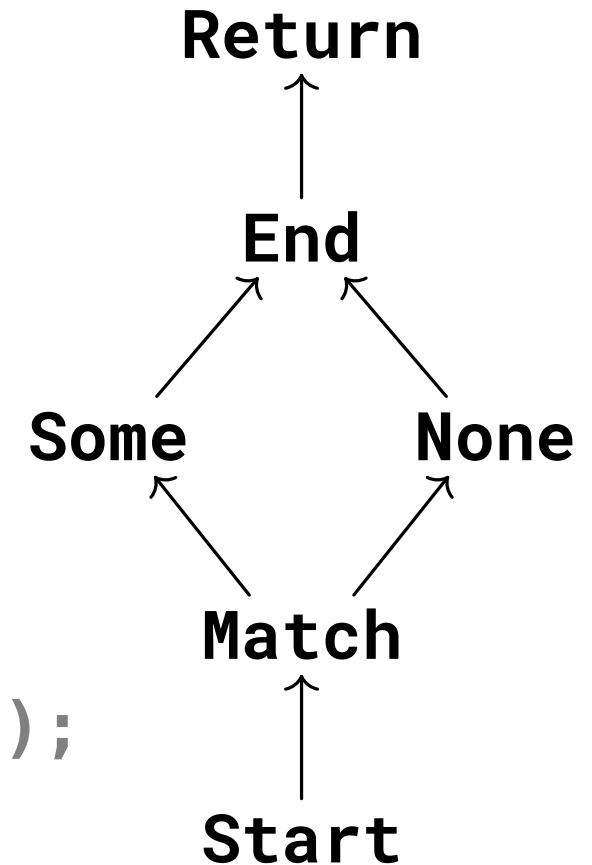
Breaking NLL

```
fn get_default<'m,K,V>(
  map: &'m mut HashMap<K,V>,
  key: K
) -> &'m mut V {
  match map.get_mut(&key) {
    Some(value) =>
      value,
    None => {
      map.insert(key, ...);
      map.get_mut(&key)
    }
  }
}
```



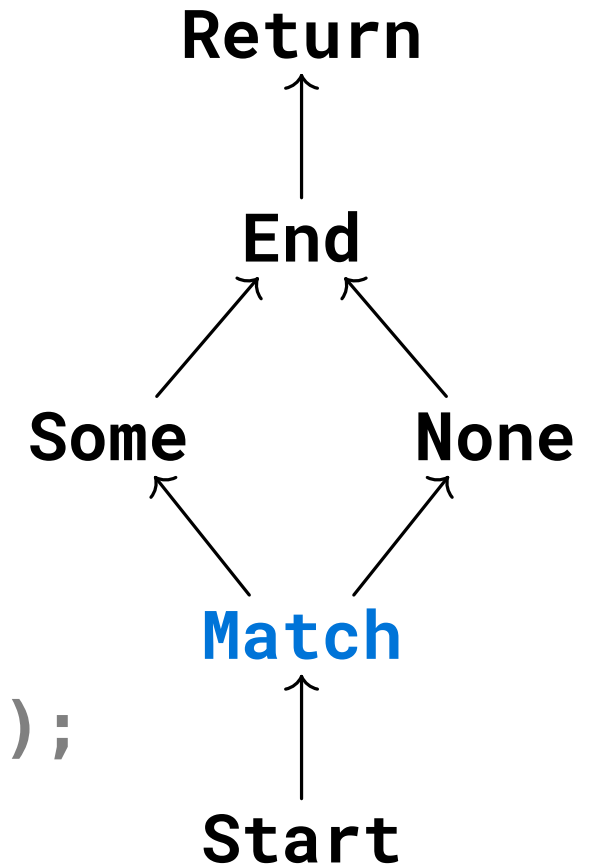
Breaking NLL

```
fn get_default<'m,K,V>(
    map: &'m mut HashMap<K,V>,
    key: K
) -> &'m mut V {
    match map.get_mut(&key) {
        Some(value) =>
            value,
        None => {
            map.insert(key, ...);
            map.get_mut(&key)
        }
    }
}
```



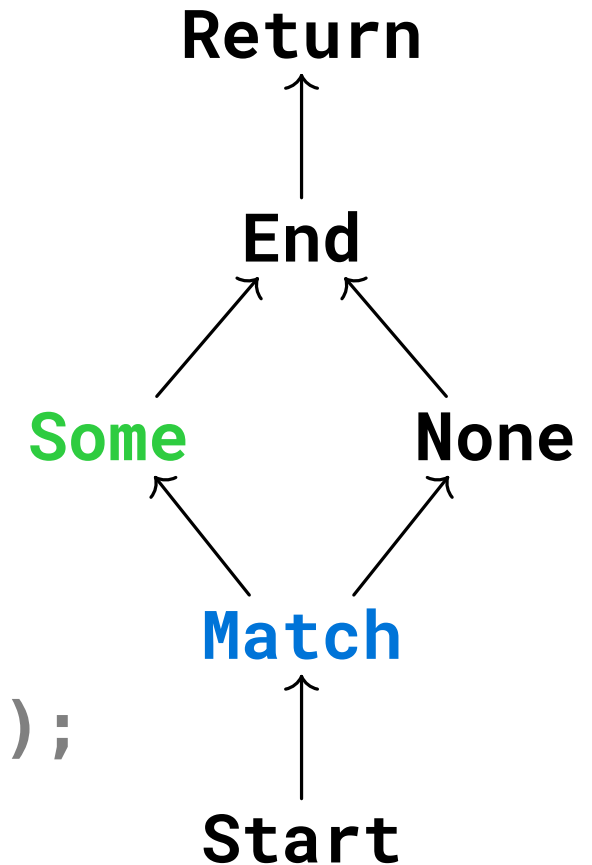
Breaking NLL

```
fn get_default<'m,K,V>(
  map: &'m mut HashMap<K,V>,
  key: K
) -> &'m mut V {
  match map.get_mut(&key) {
    Some(value) =>
      value,
    None => {
      map.insert(key, ...);
      map.get_mut(&key)
    }
  }
}
```



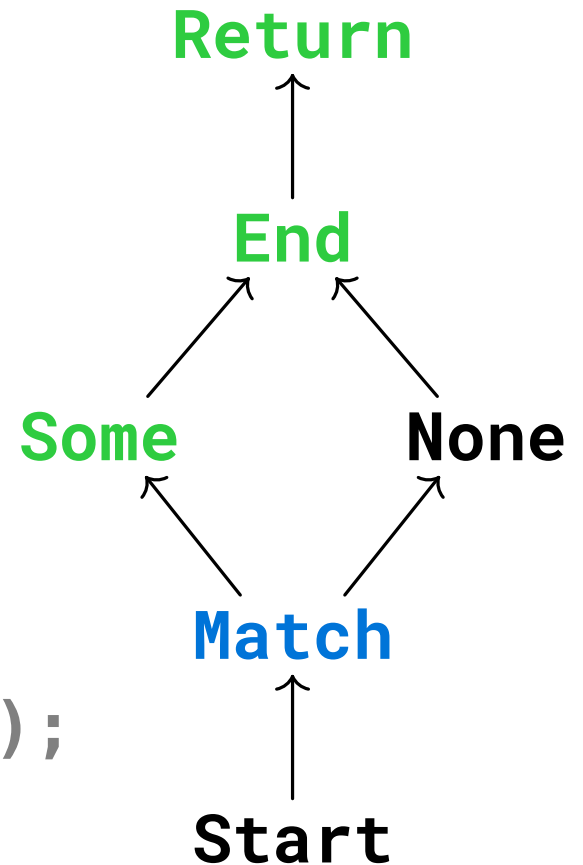
Breaking NLL

```
fn get_default<'m,K,V>(
    map: &'m mut HashMap<K,V>,
    key: K
) -> &'m mut V {
    match map.get_mut(&key) {
        Some(value) =>
            value,
        None => {
            map.insert(key, ...);
            map.get_mut(&key)
        }
    }
}
```



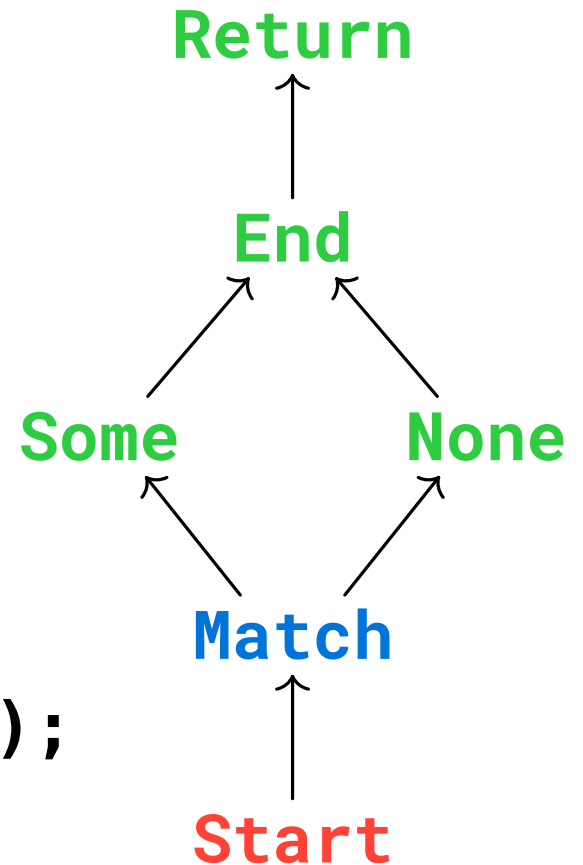
Breaking NLL

```
fn get_default<'m,K,V>(
  map: &'m mut HashMap<K,V>,
  key: K
) -> &'m mut V {
  match map.get_mut(&key) {
    Some(value) =>
      value,
    None => {
      map.insert(key, ...);
      map.get_mut(&key)
    }
  }
}
```



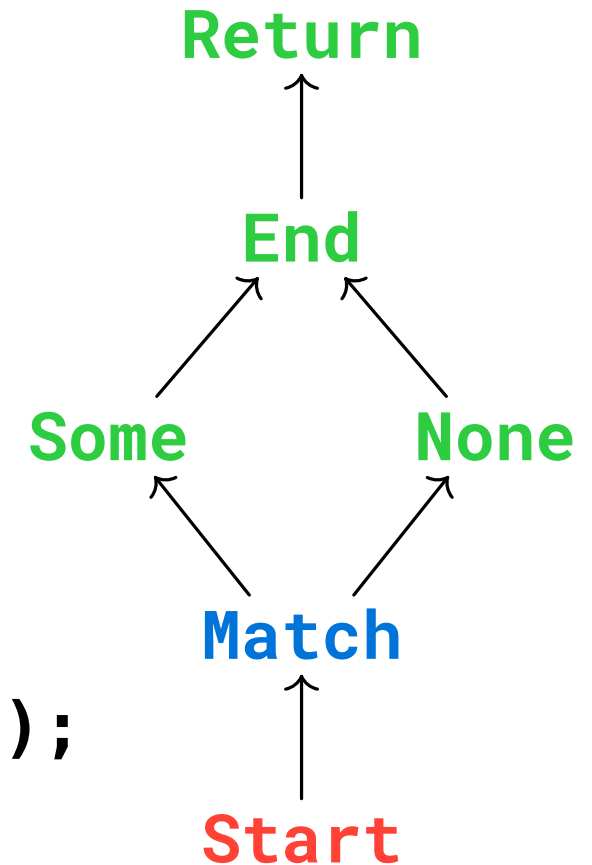
Breaking NLL

```
fn get_default<'m,K,V>(
  map: &'m mut HashMap<K,V>,
  key: K
) -> &'m mut V {
  match map.get_mut(&key) {
    Some(value) =>
      value,
    None => {
      map.insert(key, ...);
      map.get_mut(&key)
    }
  }
}
```



Breaking NLL

```
fn get_default<'m,K,V>(
  map: &'m mut HashMap<K,V>,
  key: K
) -> &'m mut V {
  match map.get_mut(&key) {
    Some(value) =>
      value,
    None => {
      map.insert(key, ...);
      map.get_mut(&key)
    }
  }
}
```

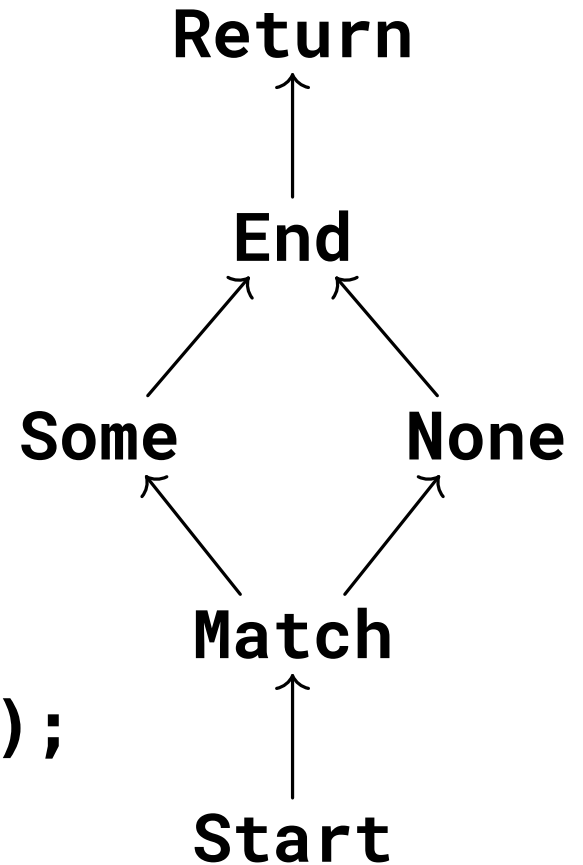


Polonius

lifetime = set of loans

Polonius

```
fn get_default<'m, K, V>(
    map: &'m mut HashMap<K, V>,
    key: K
) -> &'m mut V {
    match map.get_mut(&key) {
        Some(value) =>
            value,
        None => {
            map.insert(key, ...);
            map.get_mut(&key)
        }
    }
}
```

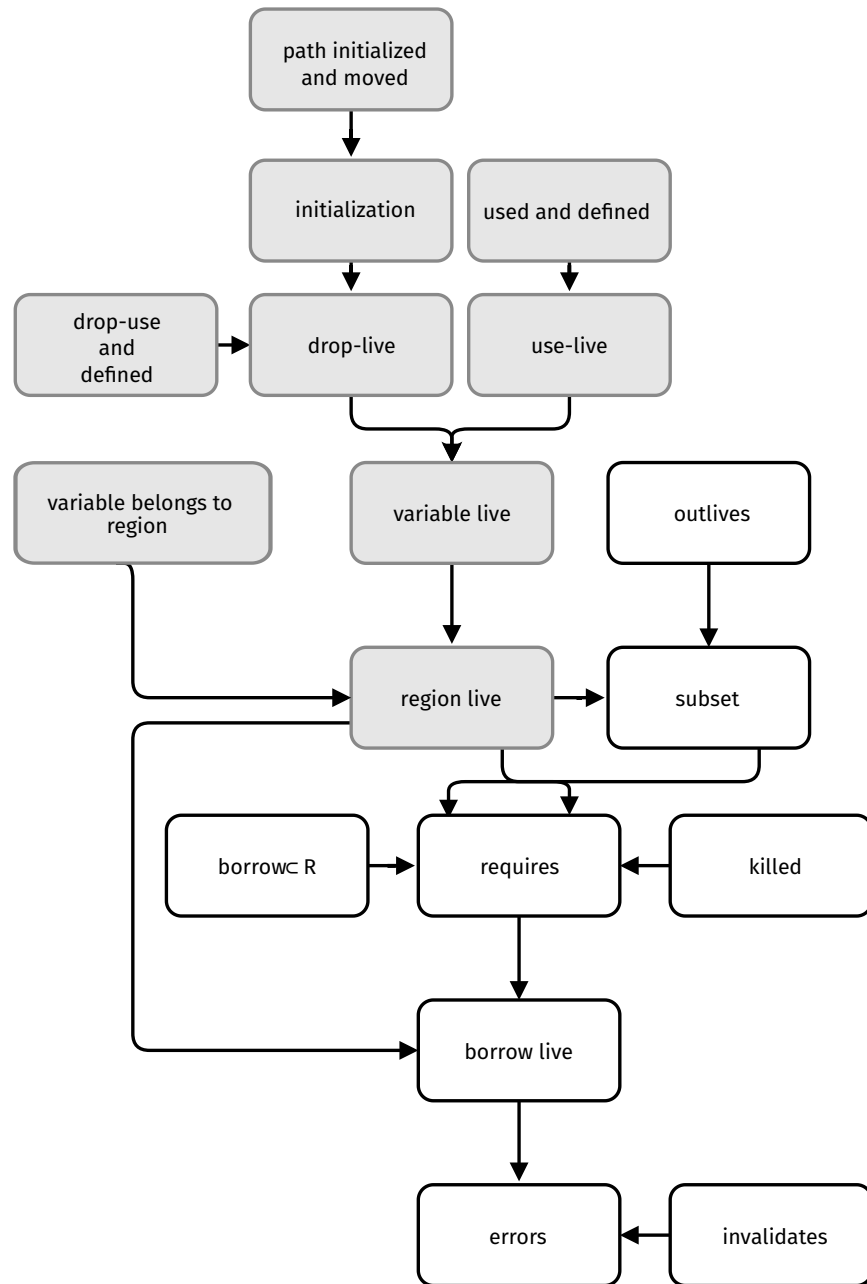


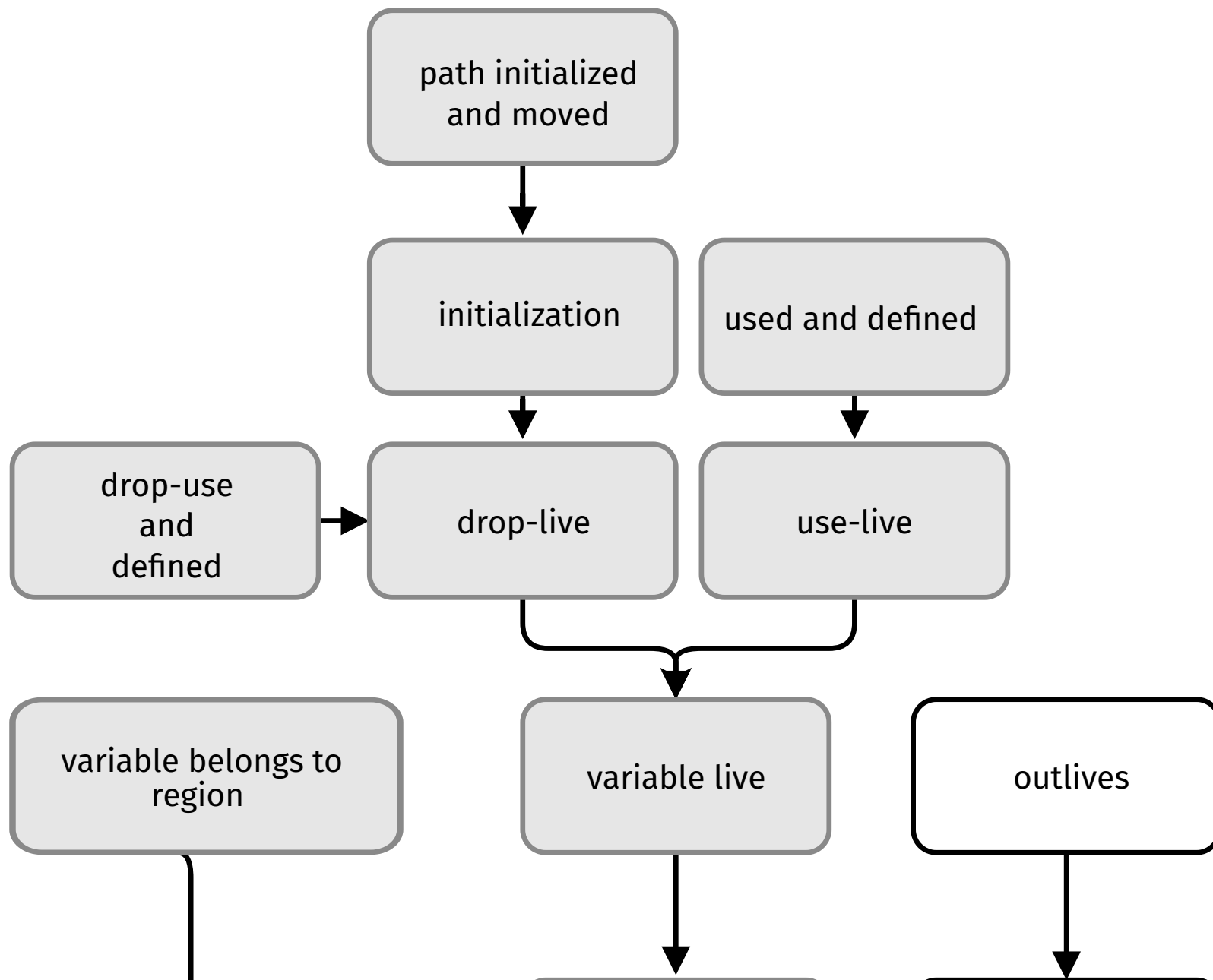
Polonius

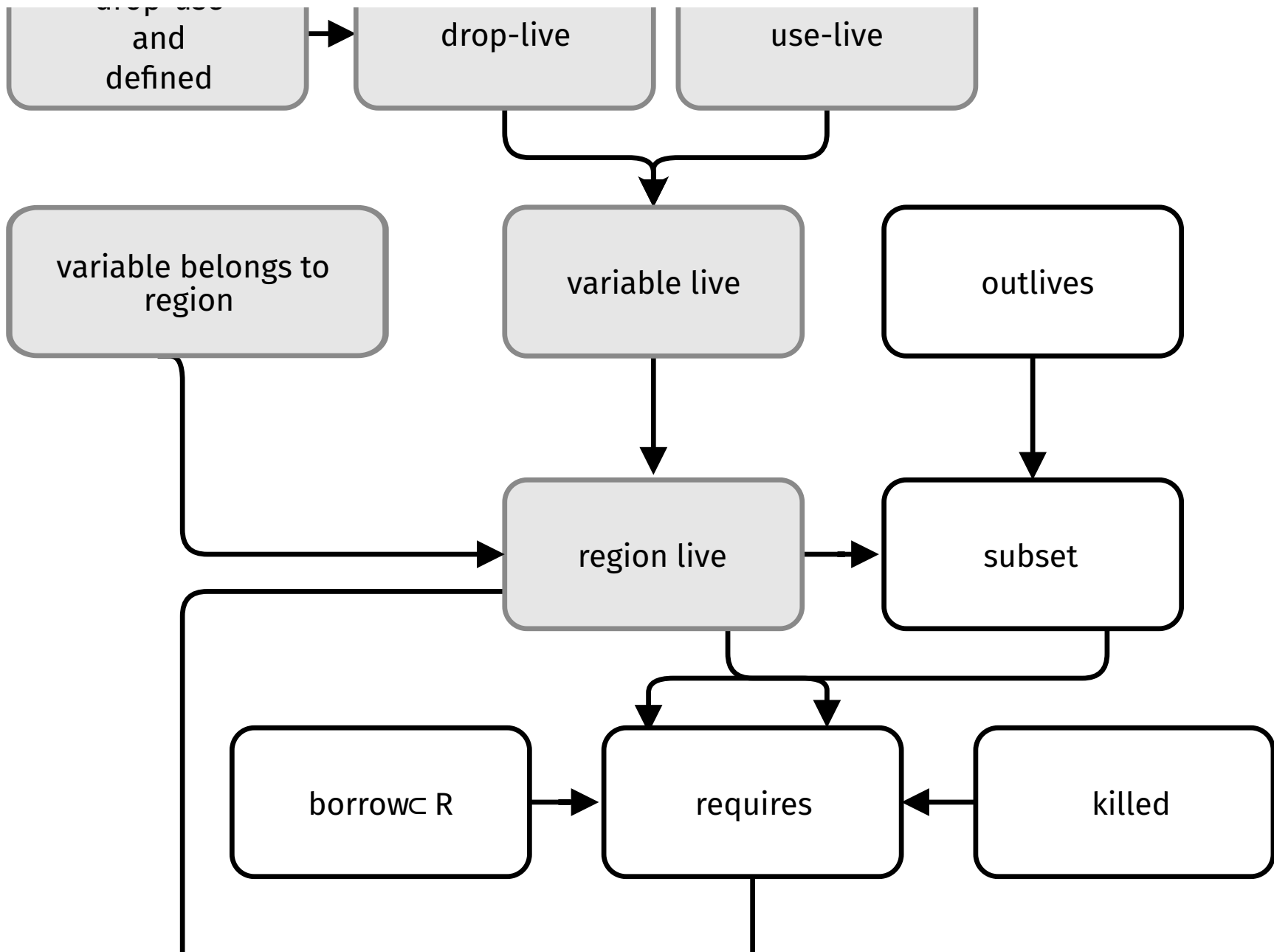
```
let r: &'0 i32 = if (cond) {  
    &x /* Loan L0 */  
} else {  
    &y /* Loan L1 */  
};
```

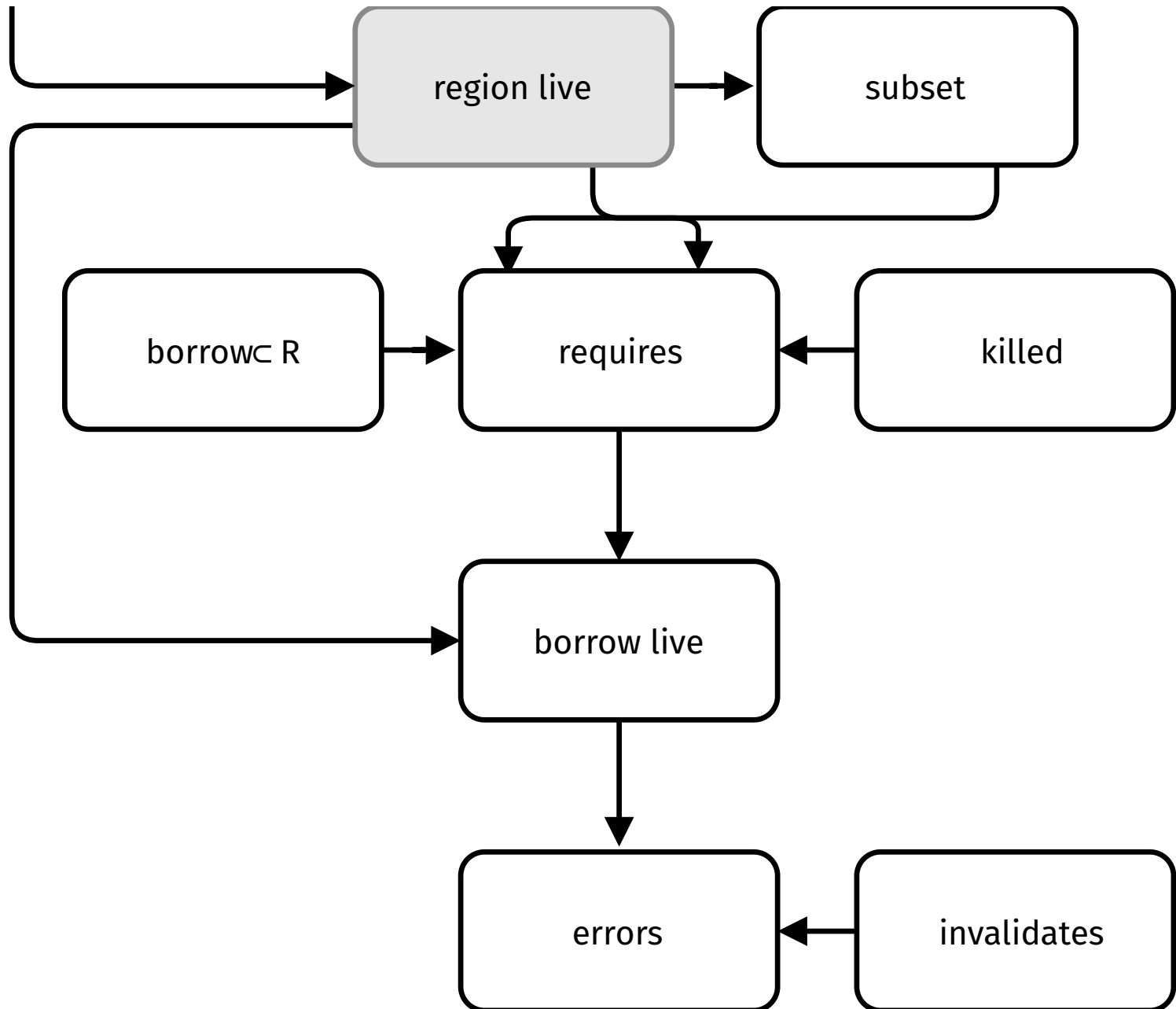
Computing!

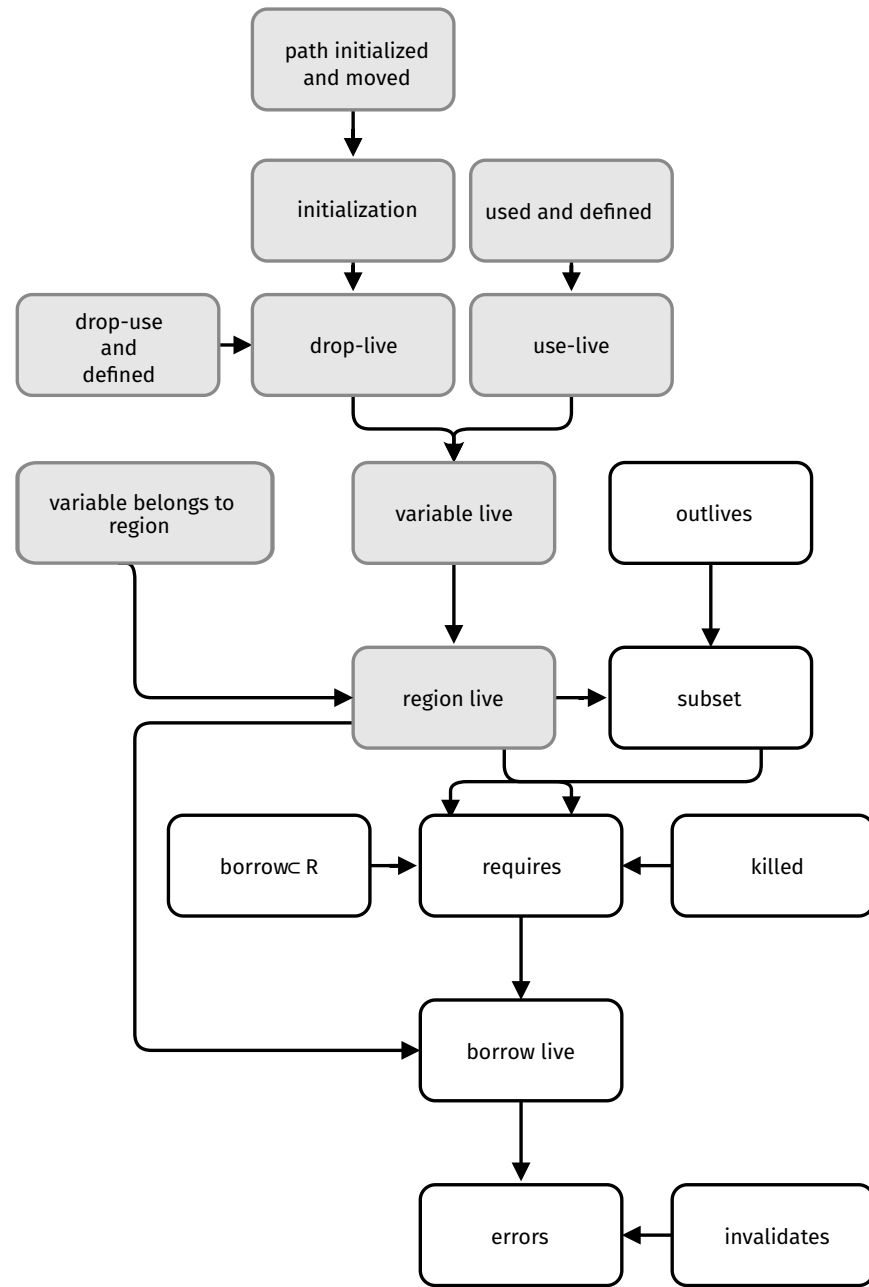
Steps of the borrow checker







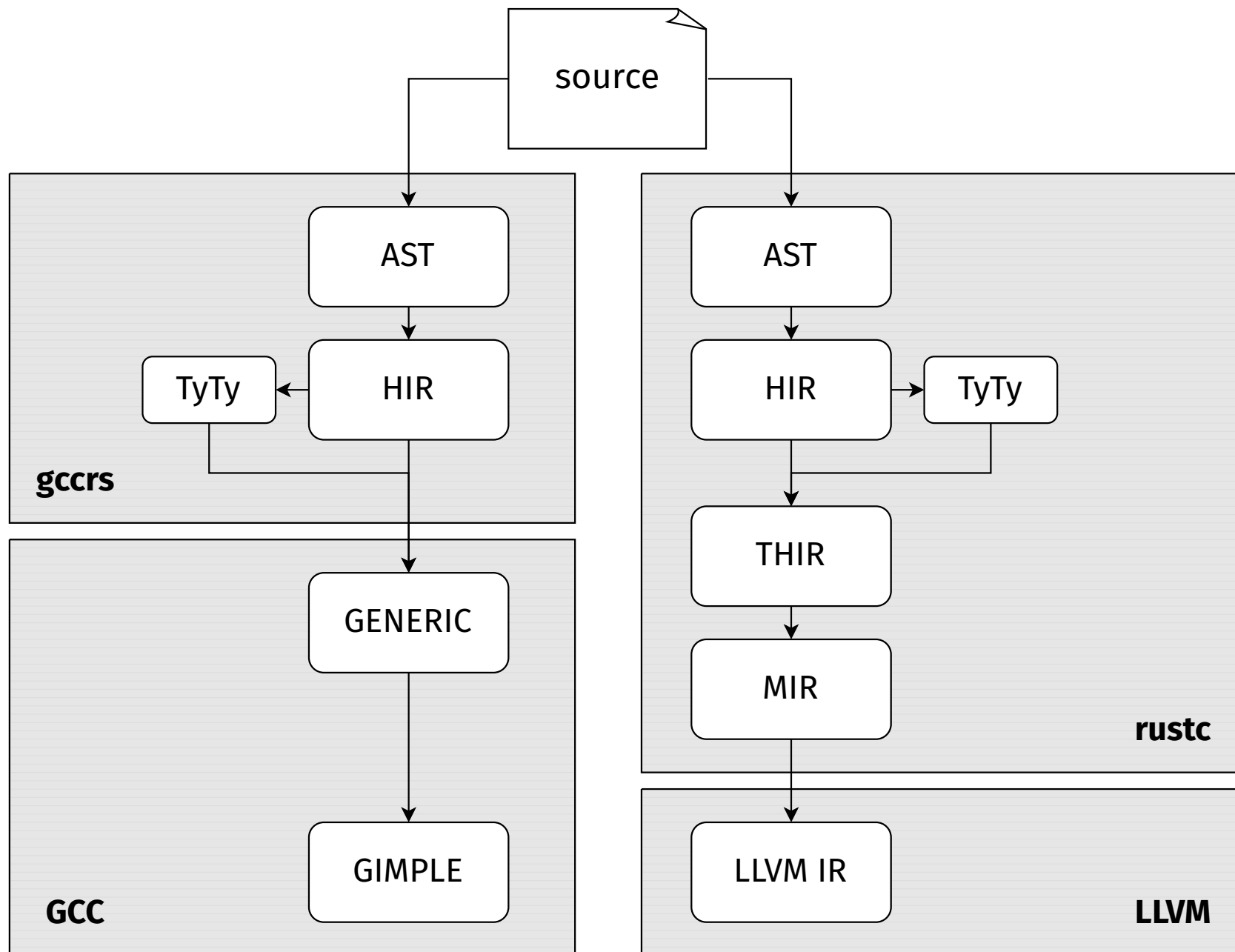




Back to Rust GCC

BIR, Variance, Polonius, Future

Rust GCC



MIR

```
struct Foo(i32);
```

```
fn foo(x: i32) -> Foo {  
    Foo(x)  
}
```

MIR

```
fn foo(_1: i32) -> Foo {  
    debug x => _1;  
    let mut _0: Foo;  
  
    bb0: {  
        _0 = Foo(_1);  
        return;  
    }  
}
```

MIR: Fibonacci

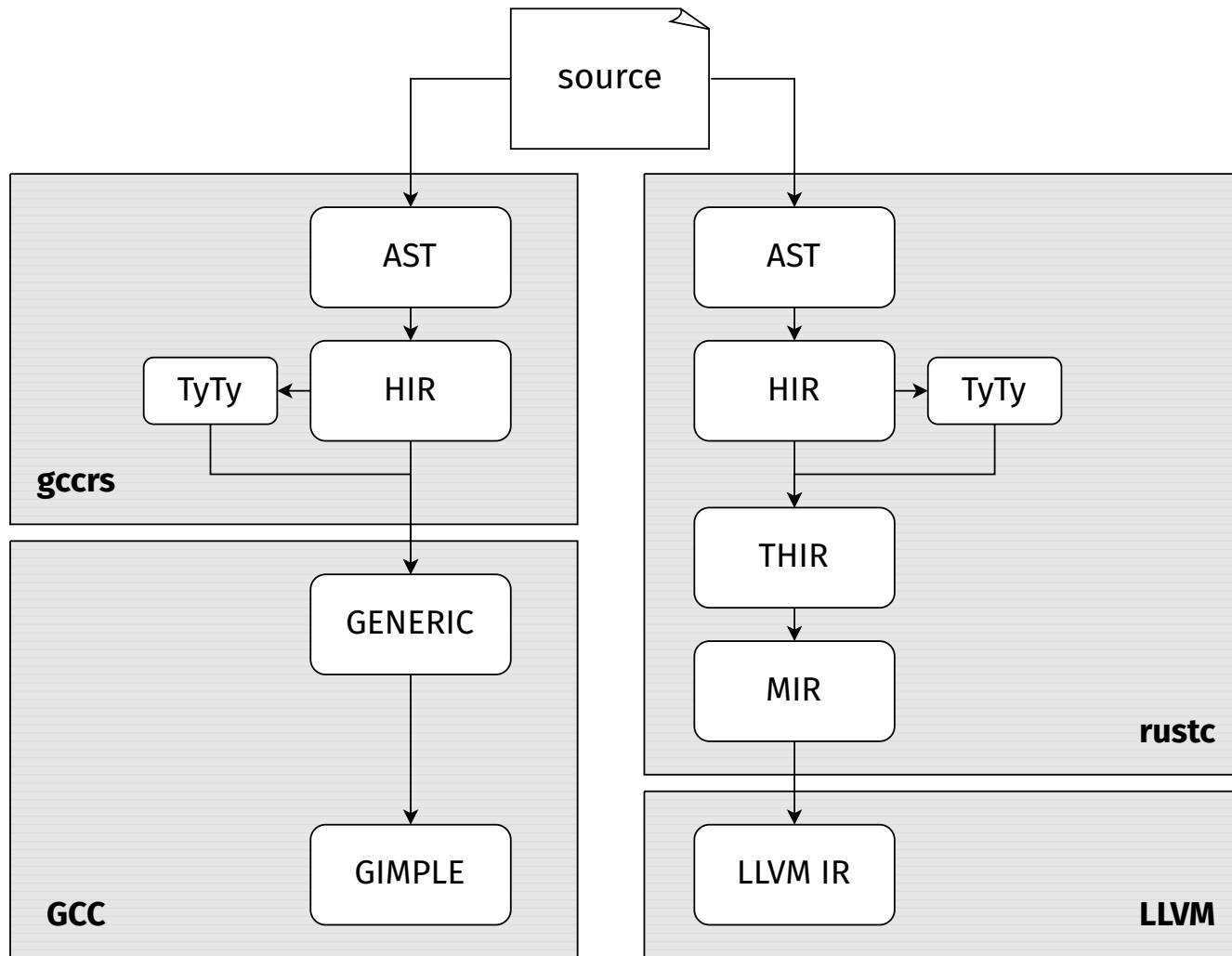
```
pub fn fib(n: u32) -> u32 {  
    if n == 0 || n == 1 {  
        1  
    } else {  
        fib(n-1) + fib(n - 2)  
    }  
}
```

MIR: Fibonacci

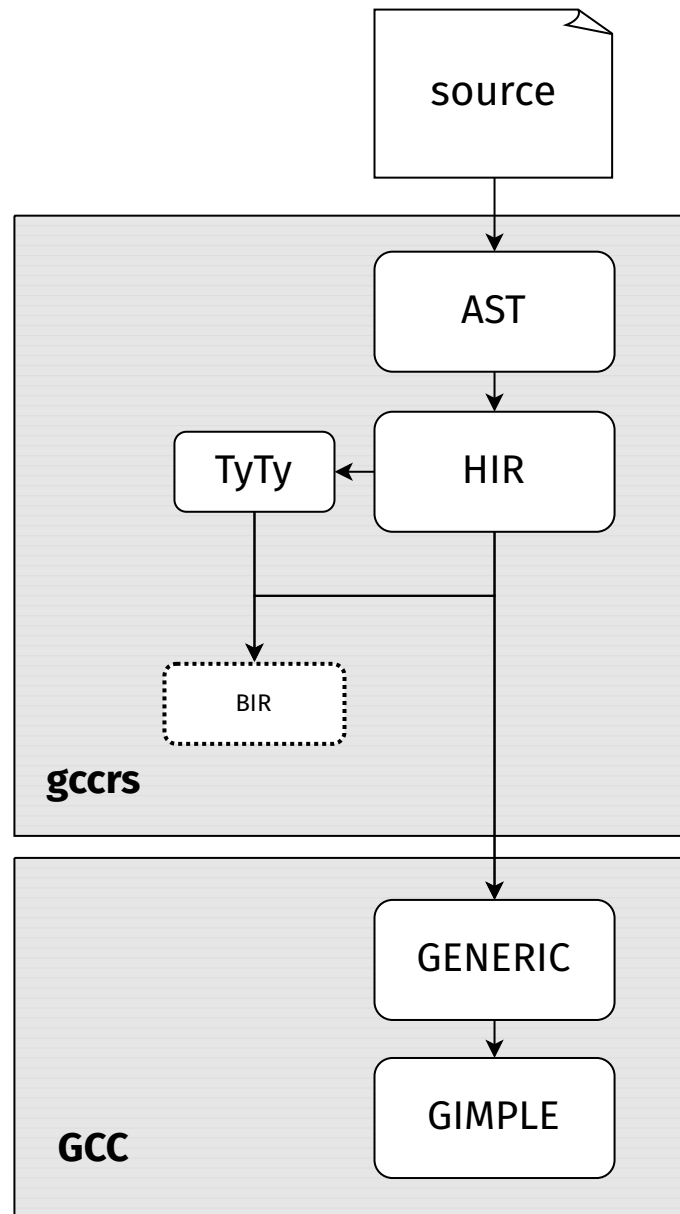
```
fn fib(_1: u32) -> u32 {
  bb0: {
    StorageLive(_2);
    StorageLive(_3);
    _3 = _1;
    _2 = Eq(move _3, const 0_u32);
    switchInt(move _2) ->
      [0: bb2, otherwise: bb1];
  }
  bb2: {
    StorageDead(_3);
    StorageLive(_4);
    StorageLive(_5);
    _5 = _1;
    _4 = Eq(move _5, const 1_u32);
    switchInt(move _4) ->
      [0: bb4, otherwise: bb3];
  }
  bb7: {
    _11 = move (_13.0: u32);
```

```
    StorageDead(_12);
    _10 = fib(move _11) ->
      [return: bb8, unwind: bb11];
  }
  bb8: {
    StorageDead(_11);
    _14 = CheckedAdd(_6, _10);
    assert(!move (_14.1: bool)) ->
      [success: bb9, unwind: bb11];
  }
  bb9: {
    _0 = move (_14.0: u32);
    StorageDead(_10);
    StorageDead(_6);
    goto -> bb10;
  }
  bb10: {
    StorageDead(_4);
    StorageDead(_2);
    return;
  }
}
```

Rust GCC



Rust GCC



BIR: Borrow Checker IR

- **basic block list**
 - **basic block**
- **place database**
- **arguments**
- **return type**
- **universal lifetimes**
- **universal lifetime constraints**

BIR: Borrow Checker IR

- **Statement**
 - **Assignment**
 - **InitializerExpr**
 - **Operator<ARITY>**
 - **BorrowExpr**
 - **AssignmentExpr (copy)**
 - **CallExpr**
 - **Switch,**
 - **Goto**
 - **Return**
 - **StorageLive, StorageDead**
 - **UserTypeAsscription**

BIR: Fibonacci

```
fn fib(_2: u32) -> u32 {  
  bb0: {  
    StorageLive(_3);  
    StorageLive(_5);  
    _5 = _2;  
    StorageLive(_6);  
    _6 = Operator(  
      move _5, const u32);  
    switchInt(move _6) ->  
      [bb1, bb2];  
  }  
  bb2: {  
    StorageLive(_8);  
    _8 = _2;  
    StorageLive(_9);  
    _9 = Operator(  
      move _8, const u32);  
    _3 = move _9;  
    goto -> bb3;  
  }  
}
```

```
bb4: {  
  _1 = const u32;  
  goto -> bb8;  
}  
  
bb5: {  
  StorageLive(_14);  
  _14 = _2;  
  StorageLive(_15);  
  _15 = Operator(  
    move _14, const u32);  
  StorageLive(_16);  
  _16 = Call(fib)(move _15) ->  
    [bb6];  
}  
  
bb8: {  
  return;  
}
```

Facts collector

<Origin, Loan, Point>

Origin

<Point, Point>

<Loan, Point>

<Origin, Origin, Point>

<Point, Loan>

<Variable, Point>

<Variable, Point>

<Variable, Point>

<Variable, Origin>

<Variable, Origin>

<Path, Path>

<Path, Variable>

<Path, Point>

<Path, Point>

<Path, Point>

<Origin, Origin>

<Origin, Loan>

loan_issued_at

universal_region

cfg_edge

loan_killed_at

subset_base

loan_invalidated_at

var_used_at

var_defined_at

var_dropped_at

use_of_var_derefs_origin

drop_of_var_derefs_origin

child_path

path_is_var

path_assigned_at_base

path_moved_at_base

path_accessed_at_base

known_placeholder_subset

placeholder

Generics and Variance

```
let a: &?1 i32;
```

```
let b: &?2 i32;
```

```
/// ...
```

```
a = b;
```

Generics and Variance

```
let a: Foo<'a, 'b, T>;  
let b: Foo<'a, 'b, T>;  
/// ...  
a = b;
```

Generics and Variance

```
let a: Foo<'a, 'b, T>;
```

```
let b: Foo<'a, 'b, T>;
```

```
/// ...
```

```
a = b;
```

```
struct Foo<'a, 'b, T> {
```

```
    x: &'a T,
```

```
    y: Bar<T>,
```

```
}
```

Bootstrapping

- Polonius is written in advanced Rust
- Similar problem with proc macros
- For now
 - Optional component compiled by *rustc*

GSoC 2024



Borrow-checking IR location support

Kushal Pal

<https://summerofcode.withgoogle.com/programs/2024/projects/DPiEgdZa>

<https://github.com/braw-lee/gsoc-2024/blob/main/README.md>

Detected Errors

Move, Subset and Loan Errors

Detected Errors

- **Mainly limited by BIR translation**
- **Detected**
 - **Move errors**
 - **Subset errors**
 - **Loan errors**

Move Errors

```
fn f() {  
    struct A {  
        i: i32,  
    }  
    let a = A { i: 1 };  
    let b = a;  
    let c = a;  
}
```

Move Errors

```
fn f() {  
    struct A { i: i32 }  
    let a = A { i: 1 };  
    let b = a;  
    let c = a;  
}
```

example_2.rs:2:1: error: Found move errors in
function f

```
2 | fn f() {  
  | ^~
```

Move Errors

```
fn f() {  
    struct A { i: i32 }  
    let a = A { i: 1 };  
    let b = a;  
    let c = a;  
}
```

example_2.rs:8:13: error: use of moved value

```
7 |         let b = a;  
  |                 ~  
  |                 |  
  |                 value moved here  
8 |         let c = a;  
  |                 ^  
  |                 |  
  |                 moved value used here
```

Loan Errors

```
fn f() {  
    let mut x = 0;  
    let y = &mut x;  
    let z = &x;  
    let w = y;  
}
```

Loan Errors

```
fn f() {  
    let mut x = 0;  
    let y = &mut x;  
    let z = &x;  
    let w = y;  
}
```

example_1.rs:2:1: error:

Found loan errors in function f

```
2 | fn f() {  
  | ^~
```


Loan Errors

```
fn f() {  
    let mut x = 0;  
    let y = &mut x;  
    let z = &x;  
    let w = y;  
}
```

example_1.rs:5:13: error: use of borrowed value

4		let y = &mut x;	
			~
			borrow occurs here
5		let z = &x;	
			^
			borrowed value used here

Subset Errors

```
fn f<'a, 'b>(
    b: bool, x: &'a u32, y: &'b u32
) -> &'a u32 {
    if b {
        y
    } else {
        x
    }
}
```

Subset Errors

```
fn f<'a, 'b>(b, x: &'a, y: &'b) -> &'a {  
    if b { y } else { x }  
}
```

example_3.rs:2:1: error:

Found subset errors in function f.

Some lifetime constraints need to be added.

```
2 | fn f<'a, 'b>(...) -> &'a u32 {  
  | ^~
```

Subset Errors

```
fn f<'a, 'b>(b, x: &'a, y: &'b) -> &'a {  
    if b { y } else { x }  
}
```

example_3.rs:2:1: error: subset error,
some lifetime constraints need to be added

```
2 | fn f<'a, 'b>(...) -> &'a u32 {  
  | ^~      ~~      ~~  
  | |      |      |  
  | |      |      lifetime defined here  
  | |      lifetime defined here  
  | subset error occurs in this function
```

Conclusion

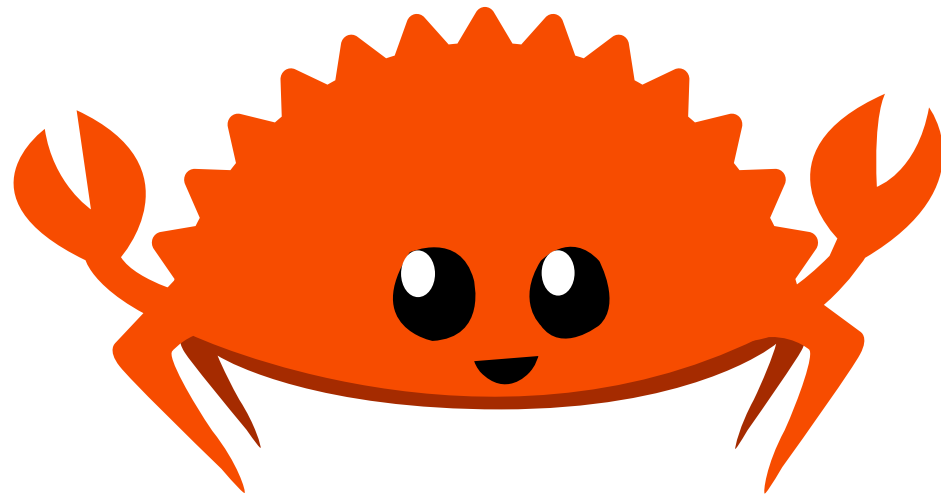
Polonius trouble, Future work

Polonius Engine Deprecated

- **Problems with over-materialization**
- **Rust Edition 2024**
 - **Polonius Algorithm**
 - **NLL Infrastructure**
- **Only minimal part of this work is bound to Polonius engine**

TODO

- **Finish translation to BIR**
 - **Match expressions**
 - **Two-phase borrowing**
 - **Drops**
- **Collect implicit type constrains**
 - **(`&'a T => T: 'a`)**
- **Improve error messages**
 - **Store additional information**
 - **Deduplicate by reason**
- **Emit crate metadata**
- **(see *my thesis [section 6.1]* for a more detailed list)**



That's all Folks!

References

Slides made with Typst and Polylux.

- DUPAK, Jakub. Memory Safety Analysis in Rust GCC. Available from: <https://dspace.cvut.cz/bitstream/handle/10467/113390/F3-DP-2024-Dupak-Jakub-thesis.pdf>
- PAL, Kushal. Google Summer of Code 2024 : Final report. Available from <https://github.com/braw-lee/gsoc-2024/blob/main/README.md>
- MATSAKIS, Niko. Non lexical lifetimes introduction. Available from: <https://smallcultfollowing.com/babysteps/blog/2016/04/27/non-lexical-lifetimes-introduction/>
- MATSAKIS, Niko. 2094-nll. In : The Rust RFC Book. Online. Rust Foundation, 2017. Available from <https://rust-lang.github.io/rfcs/2094-nll.html>
- STJERNA, Amanda. Modelling Rust's Reference Ownership Analysis Declaratively in Datalog. Online. Master's thesis. Uppsala University, 2020. Available from: <https://www.diva-portal.org/smash/get/diva2:1684081/fulltext01.pdf>
- MATSAKIS, Niko, RAKIC, Rémy and OTHERS. The Polonius Book. 2021. Rust Foundation.
- GJENGSET, Jon. Crust of Rust: Subtyping and Variance. 2022. Available from <https://www.youtube.com/watch?v=iVYWDIW71jk>
- Rust Compiler Development Guide. Online. Rust Foundation, 2023. Available from <https://rustc-dev-guide.rust-lang.org/index.html>
- TOLVA, Karen Rustad. Original Ferris.svg. Available from https://en.wikipedia.org/wiki/File:Original_Ferris.svg

This work is licensed under CC BY 4.0.

Sources are available at <https://github.com/jdupak/slides-gnu-cauldron-2024-borrowing-polonius>